



JANUARY 15, 2026

ARTIFICIAL INTELLIGENCE

DR. IRUM MATLOOB

ASSIGNMENT-3

HAJRA SARWAR
2023-BSE(5A)-022



TABLE OF CONTENTS

TASK 1: AI RAG System Implementation (Practical Component)	3
1. Environment Setup.....	3
2. Project Structure Initialization.....	4
3. Installing Dependencies.....	4
4. Setting Up OpenAI API Key.....	4
5. Vector Database Setup (Quadrant).....	5
6. Creating Vector Database Client.....	5
7. Data Loader Implementation.....	6
8. FastAPI Server Setup.....	6
9. Inngest Integration.....	7
10. PDF Ingestion Function.....	7
11. Query Function Implementation.....	8
12. Streamlit Frontend Creation.....	8
13. Testing and Final Output.....	9
 TASK 2: Conceptual Understanding (Theory Component)	9
1. What is Retrieval-Augmented Generation (RAG)?.....	10
2. Difference Between RAG and Agentic RAG.....	10
3. Purpose of RAG.....	10
4. Benefits of Using RAG in Agentic Development.....	10
 TASK 3: AI RAG Development Explanation	10
1. System Architecture.....	11
2. Component Description.....	11
3. Data Flow Process.....	11

AI RETRIEVAL-AUGMENTED GENERATION SYSTEM

TASK 1:

AI RAG SYSTEM IMPLEMENTATION

Steps:

1. Environment Setup

The development environment was prepared according to the video tutorial to ensure compatibility with a production-ready RAG system.

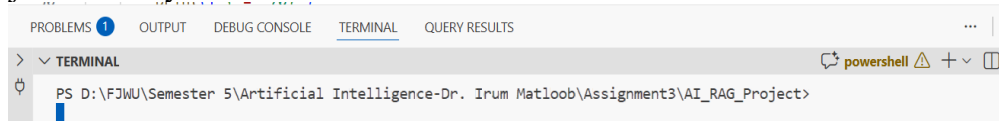
Steps:

- Installed **Python 3.10**
- Enabled *Add Python to PATH*
- Verified installation using:

`python --version`

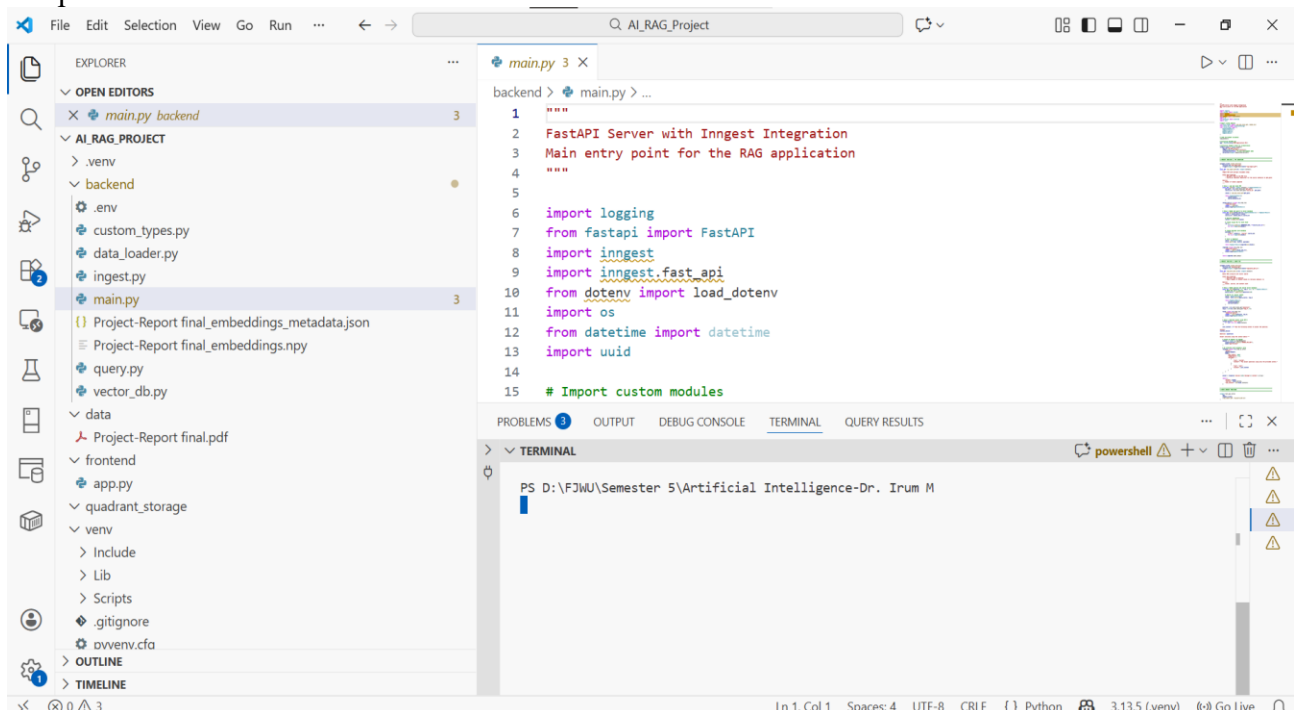
```
PS C:\Users\HP> python --version
Python 3.13.5
PS C:\Users\HP>
```

Created project directory:



2. Project Structure Initialization

A modular, production-level project structure was created to separate backend, frontend, and data components.



3. Installing Dependencies

Python dependencies were installed using pip.

Key Libraries

- FastAPI
- Inngest
- LlamaIndex

- Qdrant Client
- OpenAI
- Streamlit

```

PS D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Assignment3\AI_RAG_Project> pip list | Select-String "fastapi|inngest|llama-index|qdrant|openai|streamlit|uvicorn"
fastapi                0.115.0
inngest                0.3.5
llama-index-core       0.12.42
llama-index-readers-file 0.4.11
openai                 1.51.0
qdrant-client          1.11.0
streamlit              1.39.0
uvicorn                0.30.0
  
```

4. Setting Up OpenAI API Key

The OpenAI API key was stored securely in a .env file.

```

# OpenAI API Configuration
OPENAI_API_KEY=sk-proj-0N6DmKvumAgU2LjKUuAdu0pZYTZLSSQrW8vEX9d7iNCY-fo2fErqZaxZ-6cG5
  
```

- .env added to .gitignore
- Prevents accidental exposure

5. Vector Database Setup (Qdrant)

Qdrant was used as the vector database for semantic search.

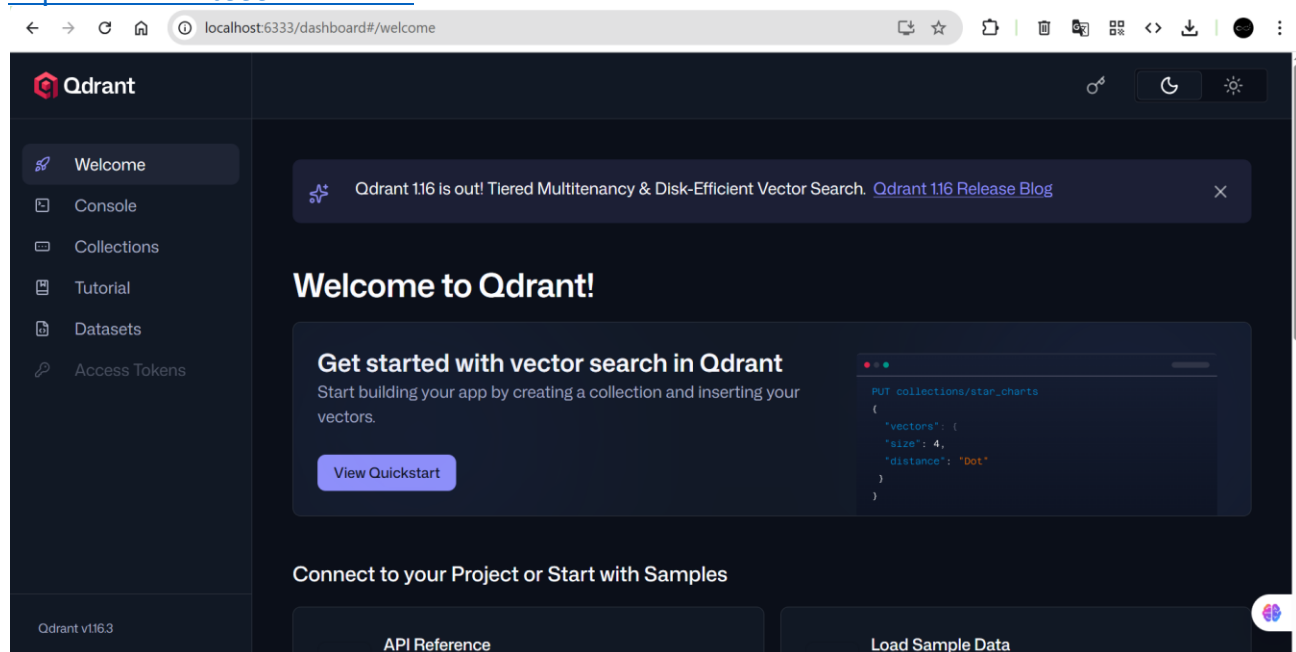
Docker Command

```

PS D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Assignment3\AI_RAG_Project> docker ps --filter "name=qdrant_rag_db"
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
4824299067e6   qdrant/qdrant  "./entrypoint.sh"       34 seconds ago Up 32 seconds 0.0.0.0:6333->6333/tcp, [::]:6333->6333/tcp
qdrant_rag_db
PS D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Assignment3\AI_RAG_Project>
  
```

Verified at:

<http://localhost:6333/dashboard>

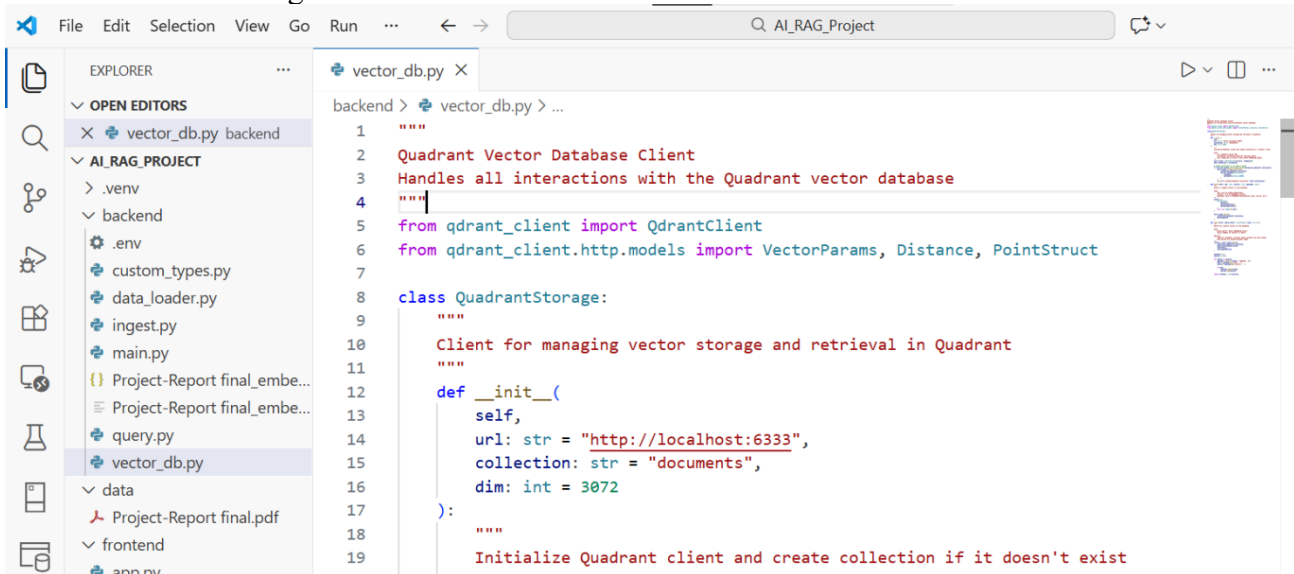


6. Creating Vector Database Client

A dedicated client module was created to manage vector operations.

Features

- Auto collection creation
- Cosine similarity
- 3072-dimension vectors
- Metadata storage



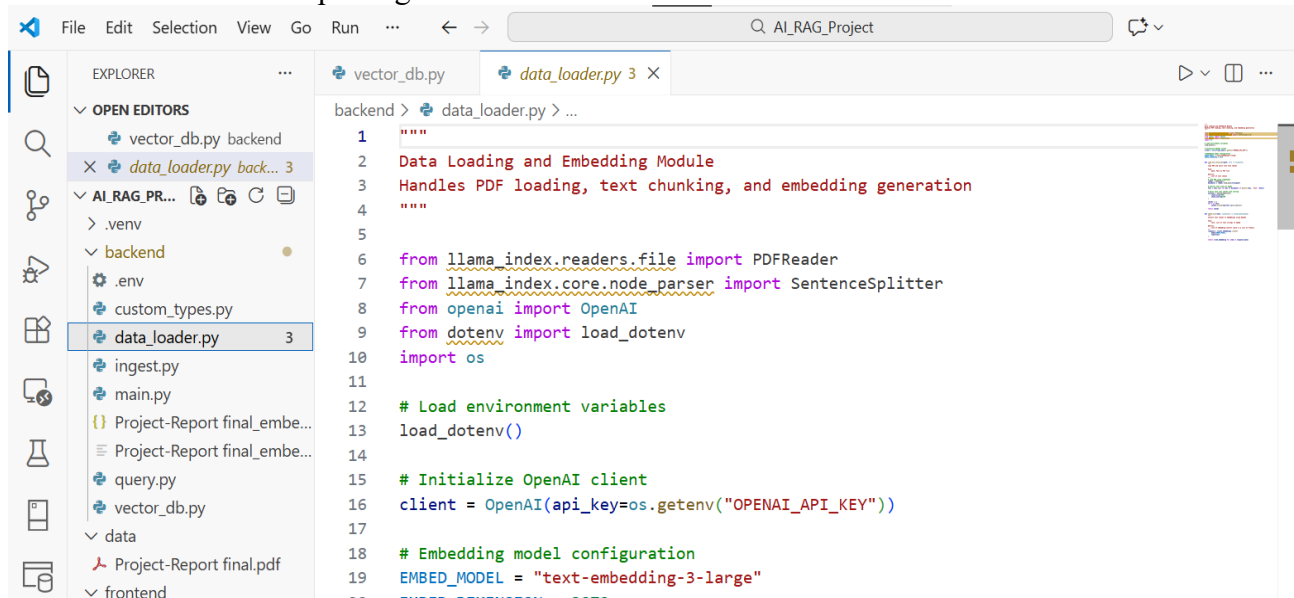
```
1 """
2 Quadrant Vector Database Client
3 Handles all interactions with the Quadrant vector database
4 """
5 from qdrant_client import QdrantClient
6 from qdrant_client.http.models import VectorParams, Distance, PointStruct
7
8 class QuadrantStorage:
9     """
10     Client for managing vector storage and retrieval in Quadrant
11     """
12     def __init__(
13         self,
14         url: str = "http://localhost:6333",
15         collection: str = "documents",
16         dim: int = 3072
17     ):
18         """
19         Initialize Quadrant client and create collection if it doesn't exist
20         """
```

7. Data Loader Implementation

PDF loading, chunking, and embedding generation were handled using LlamaIndex and OpenAI.

Chunking Strategy

- Chunk size: 1000
- Overlap: 200
- Sentence-based splitting



```
1 """
2 Data Loading and Embedding Module
3 Handles PDF loading, text chunking, and embedding generation
4 """
5
6 from llama_index.readers.file import PDFReader
7 from llama_index.core.node_parser import SentenceSplitter
8 from openai import OpenAI
9 from dotenv import load_dotenv
10 import os
11
12 # Load environment variables
13 load_dotenv()
14
15 # Initialize OpenAI client
16 client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
17
18 # Embedding model configuration
19 EMBED_MODEL = "text-embedding-3-large"
20 EMBED_DIMENSION = 3072
```

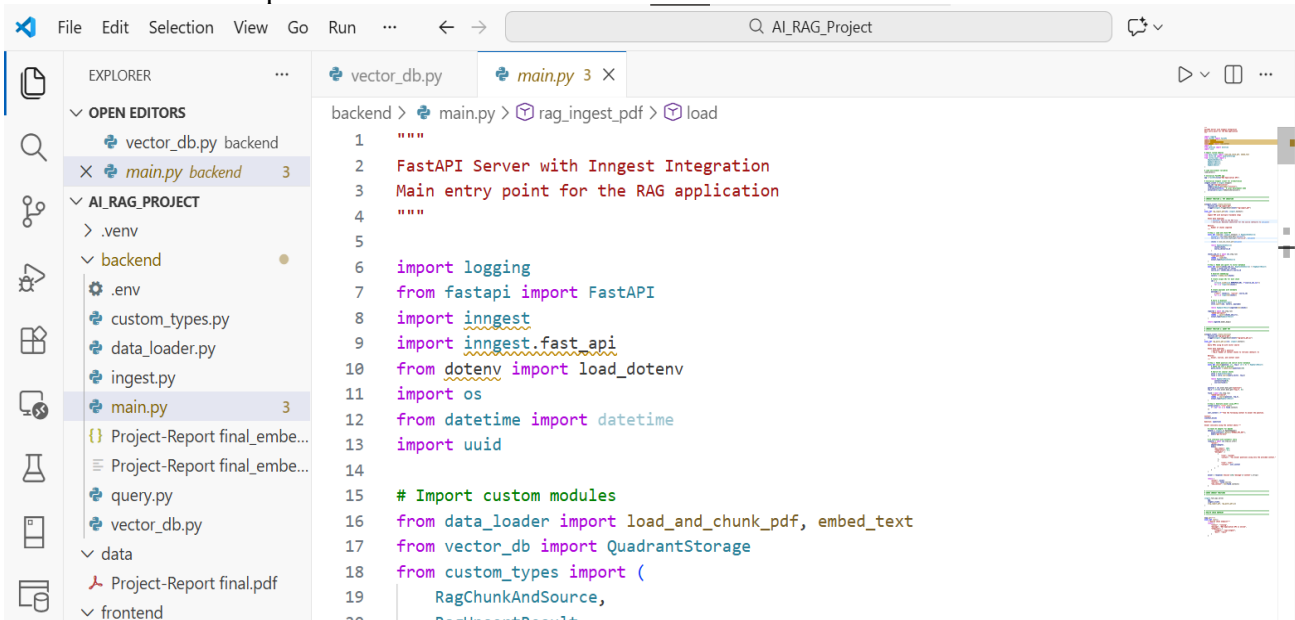
8. FastAPI Server Setup

FastAPI was configured as the backend server and integrated with Inngest.

Key Points

- Central entry point (main.py)
- Inngest endpoint auto-generated

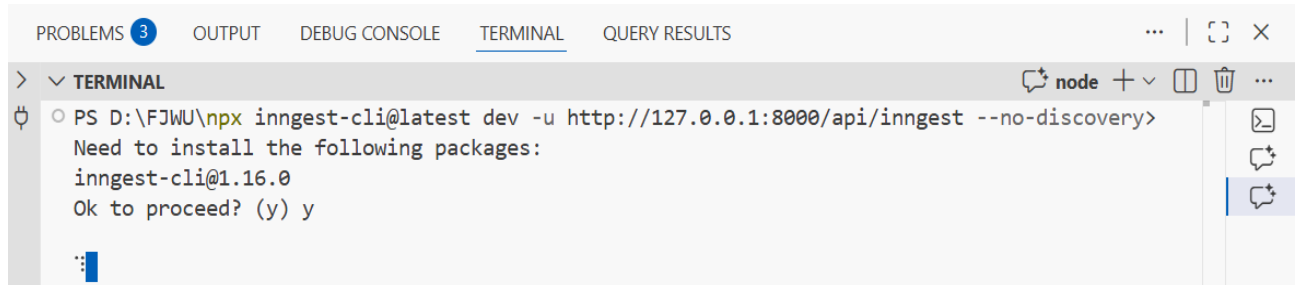
- Local development mode enabled



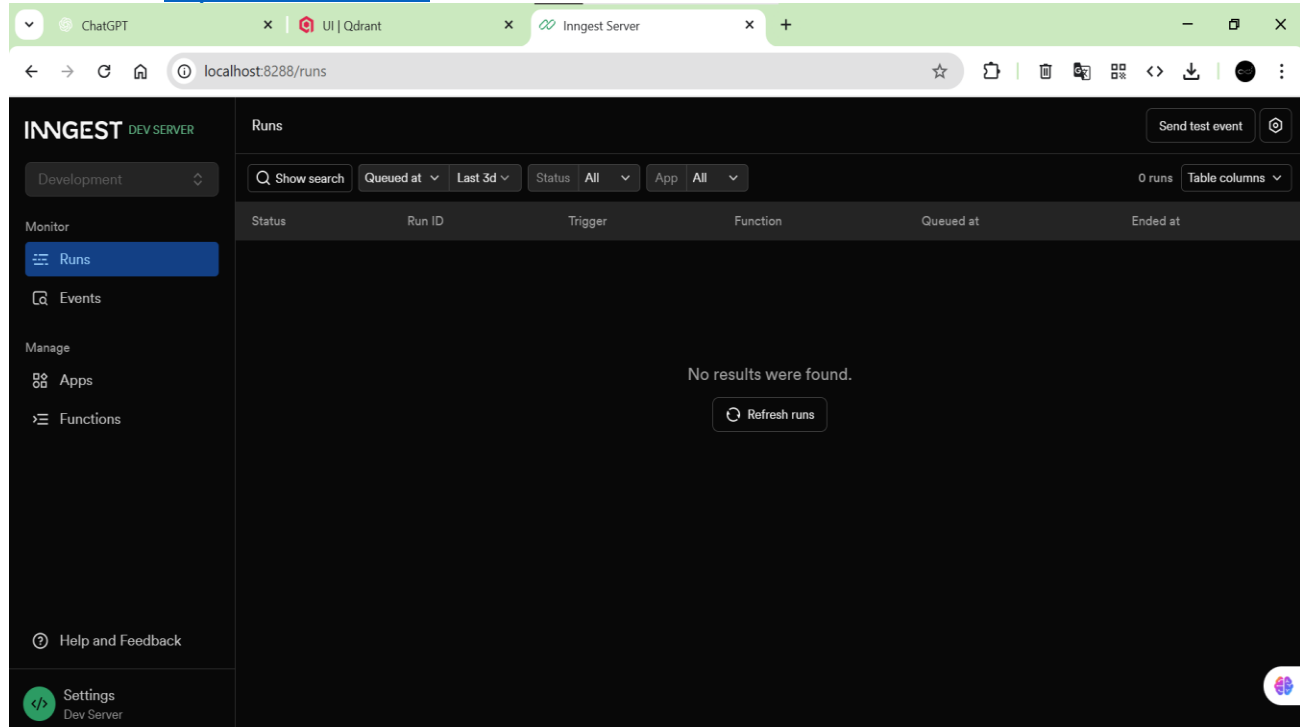
9. Inngest Integration

Inngest was used for orchestration and observability.

Commands



Dashboard: <http://localhost:8288>

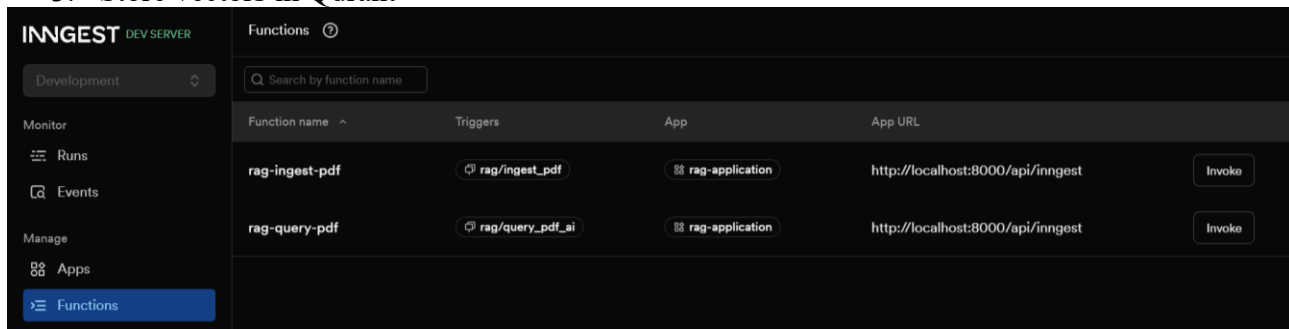


10. PDF Ingestion Function

PDF ingestion was implemented as an Inngest function with observable steps.

Steps

1. Load and chunk PDF
2. Embed chunks
3. Store vectors in Qdrant



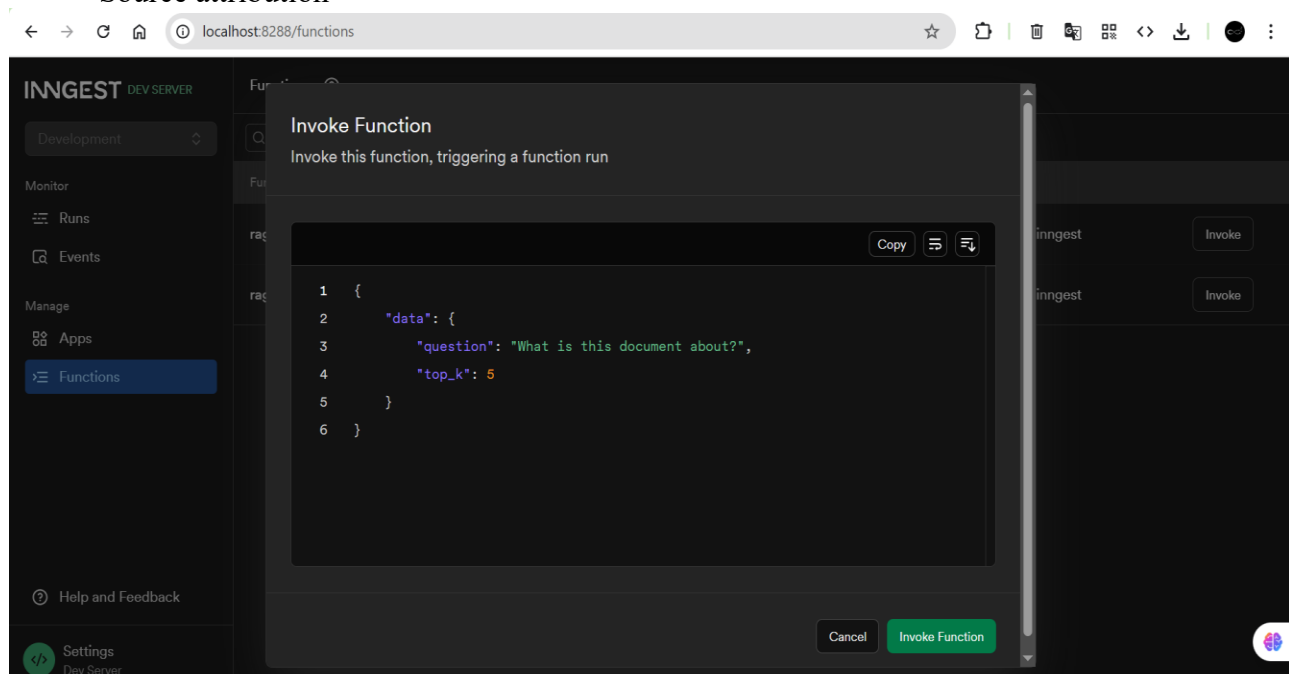
Function name	Triggers	App	App URL	
rag-ingest-pdf	rag/ingest_pdf	rag-application	http://localhost:8000/api/inngest	Invoke
rag-query-pdf	rag/query_pdf_ai	rag-application	http://localhost:8000/api/inngest	Invoke

11. Query Function Implementation

The query function retrieves relevant chunks and generates answers using GPT-4o-mini.

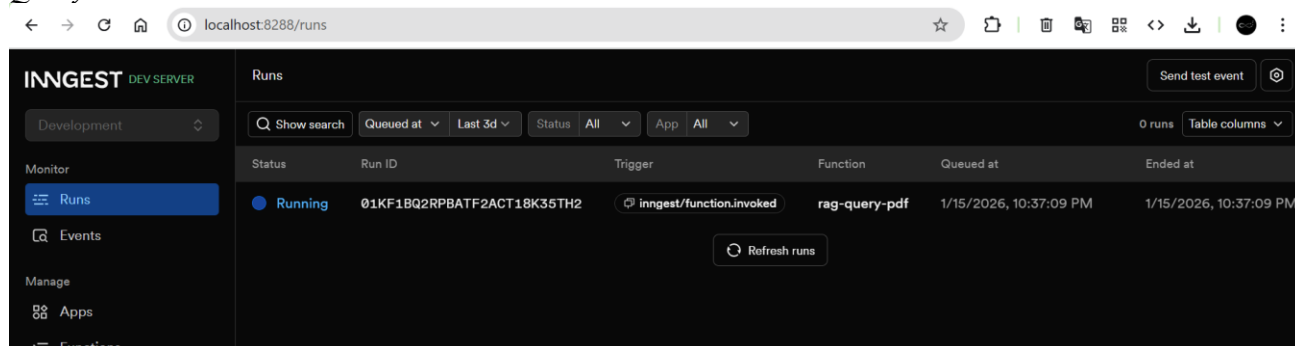
Process

- Embed query
- Vector search
- Context-based generation
- Source attribution



```
1 {
2   "data": {
3     "question": "What is this document about?",
4     "top_k": 5
5   }
6 }
```

Query execution:



Status	Run ID	Trigger	Function	Queued at	Ended at
Running	01KF1BQ2RPBATHF2ACT18K35TH2	inngest/function.invoked	rag-query-pdf	1/15/2026, 10:37:09 PM	1/15/2026, 10:37:09 PM

The screenshot shows the Inngest Dev Server interface at localhost:8288/runs. The left sidebar contains navigation options: Monitor (Runs, Events), Manage (Apps, Functions), and Settings (Dev Server). The main area displays a table of runs. One run is in the 'Running' state with ID '01KF1BQ2RPBATE2ACT18K35TH2'. Below the table, a trace view shows the execution steps: 'Run' (15.4s), 'embed...' (30.9s), and 'Atte...' (46.3s). The right panel shows the function details for 'rag-query-pdf', including the event name 'ingest/function.invoked', event ID '01KF1BQ2NDWMG7PSSX94607R3V', and the function payload.

Failed due to API was paid, then used another API:

The screenshot shows the Inngest Dev Server interface at localhost:8288/runs. The left sidebar is the same as the previous screenshot. The main area displays a table of runs. One run is in the 'Completed' state with ID '01KEA3PWDX75RZ1R'. Below the table, the function details for 'RAG: Ingest PDF' are shown, including the event name 'ingest/function.invoked' and the function payload.

12. Streamlit Frontend Creation

A Streamlit UI was created for PDF upload and Q&A.

Features

- PDF upload
- Interactive chat
- Source display

The screenshot shows the AI RAG Chatbot Streamlit UI at localhost:8501. The interface is dark-themed and includes a sidebar with 'Upload Documents' and 'Settings' sections. The main area features the 'AI RAG Chatbot' title, a description 'Upload PDF documents and ask questions about their content', and an 'Ask Questions' input field. The footer indicates 'AI RAG System • Powered by OpenAI, Quadrant & Inngest'.

13. Testing and Final Output

All components were tested end-to-end.

Verified

- PDF ingestion
- Vector storage
- Accurate answers
- Retry mechanism

 Screenshots 17–23: Testing results

TASK 2:

1. What is Retrieval-Augmented Generation (RAG)?

RAG combines **information retrieval** with **language generation** to produce accurate, source-grounded answers.

Three Phases

1. Retrieval
2. Augmentation
3. Generation

Why RAG?

- Reduces hallucinations
- Enables up-to-date knowledge
- Supports source citation

2. Difference Between RAG and Agentic RAG

Aspect	Traditional RAG	Agentic RAG
Workflow	Fixed	Adaptive
Reasoning	Single-step	Multi-step
Tool Use	Limited	Multiple tools
Autonomy	Low	High

3. Purpose of RAG

- Overcomes LLM knowledge cutoff
- Improves factual accuracy
- Enables domain-specific knowledge
- Allows source verification

4. Benefits of Using RAG in Agentic Development

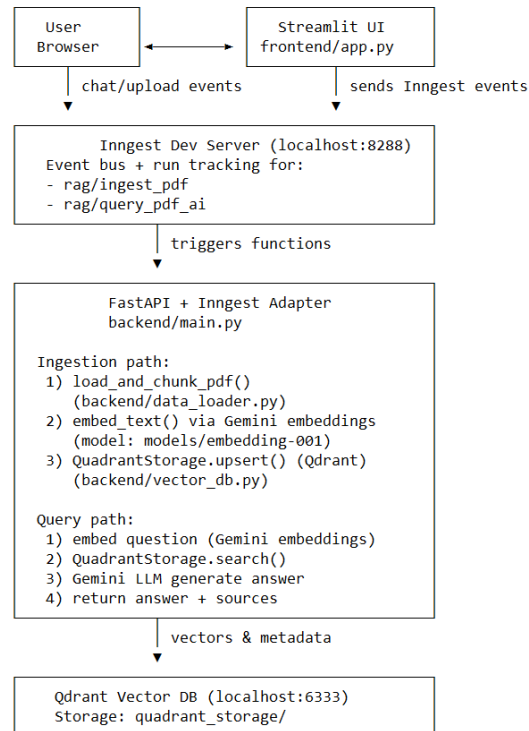
- Evidence-based decisions
 - Iterative retrieval
 - Autonomous problem solving
 - Enterprise scalability
-

TASK 3:

AI RAG DEVELOPMENT EXPLANATION

1. System Architecture

The system follows a **production-ready, modular architecture** integrating frontend, backend, orchestration, vector storage, and LLM services.



2. Component Description

Key Components

- PDF Documents (Knowledge Base)
- OpenAI Embedding Model
- Qdrant Vector Database
- Retriever
- GPT-4o-mini
- Inngest Orchestration
- Streamlit Frontend

Each component plays a distinct role in ensuring accuracy, scalability, and reliability.

3. Data Flow Process

Ingestion Flow

PDF → Chunking → Embedding → Vector Storage

Query Flow

Question → Retrieval → Context → LLM → Answer

Error Handling

- Automatic retries
- Logging
- Observability via dashboard

CONCLUSION

This assignment successfully demonstrates a **production-ready AI RAG system** with:

- Complete ingestion-to-generation pipeline
- Reliable orchestration using Inngest
- Accurate semantic search with Qdrant
- Grounded answers using GPT-4
- Clean and user-friendly frontend

The project reflects **strong practical implementation** and **clear theoretical understanding** of modern RAG systems.

REFERENCES

1. *How to Build a Production-Ready RAG AI Agent in Python* – YouTube
 2. Inngest Documentation
 3. Qdrant Documentation
 4. OpenAI API Documentation
-